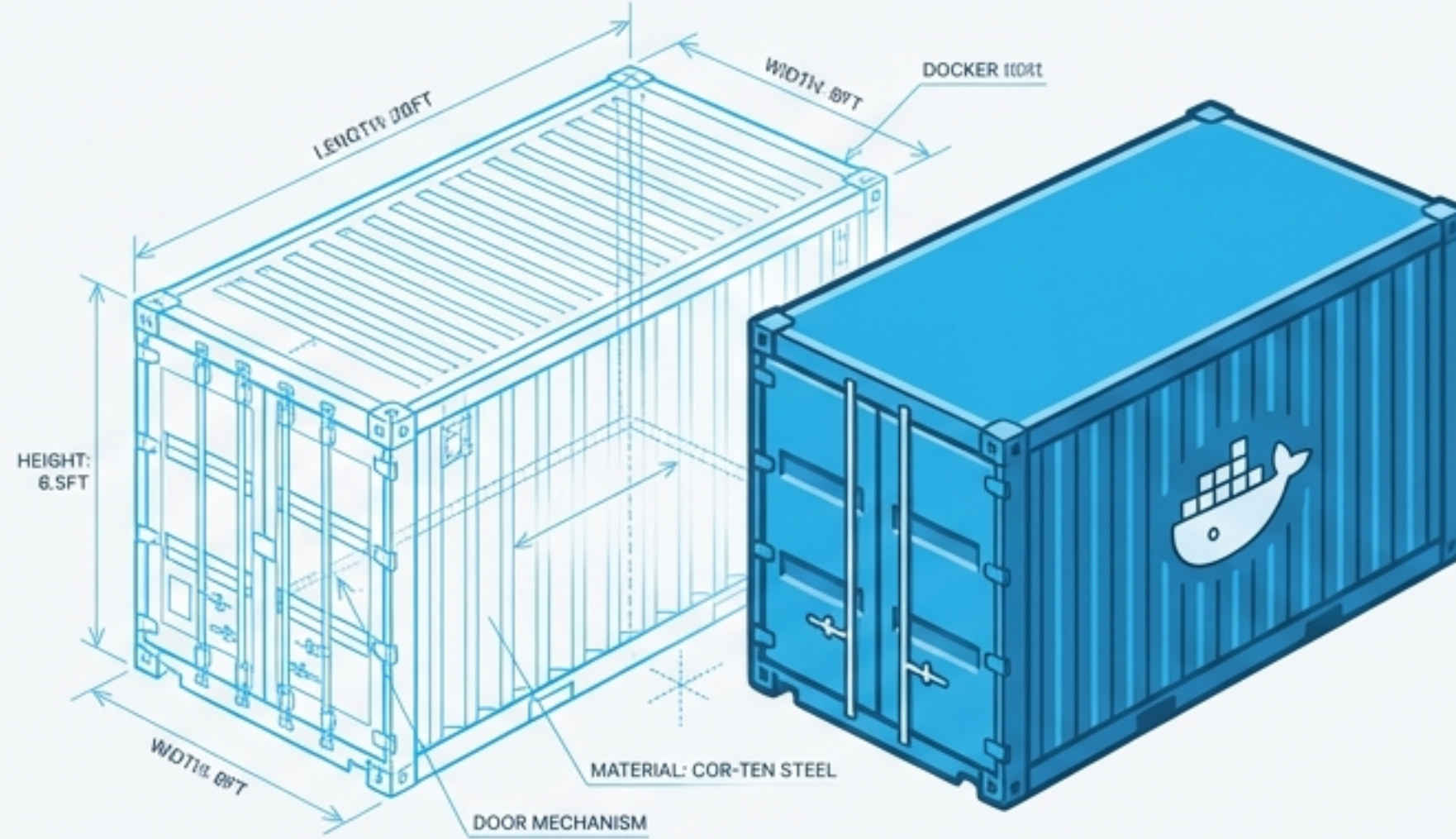




Devenez l'Architecte de Vos Conteneurs

Créez votre première image personnalisée avec un [Dockerfile](#)

Notre Mission : La Reproductibilité

Dans le TP1, nous avons utilisé une image existante. Aujourd'hui, nous allons plus loin : nous allons construire notre propre image. L'objectif est de créer un environnement PHP personnalisé, fiable et reproductible à l'infini grâce à un `Dockerfile`.

Objectifs du TP



- Comprendre le rôle fondamental d'un `Dockerfile`.



- Construire une image personnalisée basée sur `php:8.2-apache`.



- Y ajouter nos propres outils (`nano`, `iproute2`).



- Copier notre code (`index.php`) directement dans l'image.



- Lancer, vérifier et mettre à jour notre application conteneurisée.

Les Trois Piliers : La Recette, le Modèle, l'Instance



`Dockerfile`

Le fichier texte contenant les instructions. C'est la **recette** de fabrication de notre environnement.



Image

Le résultat de la construction. C'est un **modèle** immuable et portable contenant notre application et ses dépendances.



Conteneur

Une **instance** vivante de notre image. C'est le processus en cours d'exécution.

Étape 1 : Préparer le Terrain

Créons notre espace de travail et notre première page web.

Commandes

```
# Créer le répertoire du projet et s'y placer
mkdir tp-docker-dockerfile
cd tp-docker-dockerfile

# Créer le fichier index.php avec un message de
bienvenue
echo "<?php echo '<h1>Bonjour depuis mon image Docker
personnalisée 🐳 </h1>'; ?>" > index.php
```

Structure

```
📁 tp-docker-dockerfile/
└── 📄 index.php
```


Étape 2 : Rédiger la Recette

Créez un fichier nommé `Dockerfile` (sans extension) et ajoutez le contenu suivant.

```
# Image de base avec Apache + PHP
FROM php:8.2-apache

# Installer des outils utiles pour le débogage
RUN apt update && apt install -y nano iproute2

# Copier notre page PHP dans le dossier web d'Apache
COPY index.php /var/www/html/index.php

# Commande à exécuter au lancement pour démarrer le serveur
CMD ["apache2ctl", "-D", "FOREGROUND"]
```


Anatomie de notre `Dockerfile`

FROM

Rôle Choisit l'image de départ. C'est la fondation sur laquelle nous construisons.

Exemple `FROM php:8.2-apache`

RUN

Rôle Exécute des commandes *pendant la construction* de l'image (ex: installer des paquets).

Exemple `RUN apt update && apt install -y nano iproute2`

COPY

Rôle Ajoute des fichiers depuis notre machine (le contexte de build) **dans l'image**.

Exemple

`COPY index.php /var/www/html/index.php`

CMD

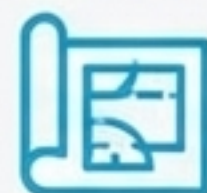
Rôle Définit la commande par défaut qui sera exécutée **au lancement d'un conteneur** basé sur cette image.

Exemple

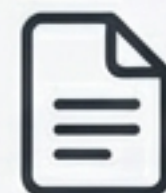
`CMD ["apache2ctl", "-D", "FOREGROUND"]`

Étape 3 : Donner Vie à l'Image

Transformons notre `Dockerfile` en une image portable et réutilisable.



Dockerfile



index.php



La Commande de Construction

```
# Assurez-vous d'être dans le dossier tp-docker-dockerfile  
docker build -t mon-php-image .
```

"Tag" ou nomme notre image
pour la retrouver facilement.



Indique que le contexte de build (l'endroit où
se trouvent le Dockerfile et les fichiers à
copier) est le répertoire courant.



Résultat Attendu

```
...  
...Successfully tagged mon-php-image:latest
```


Étape 4 : Lancer Notre Création

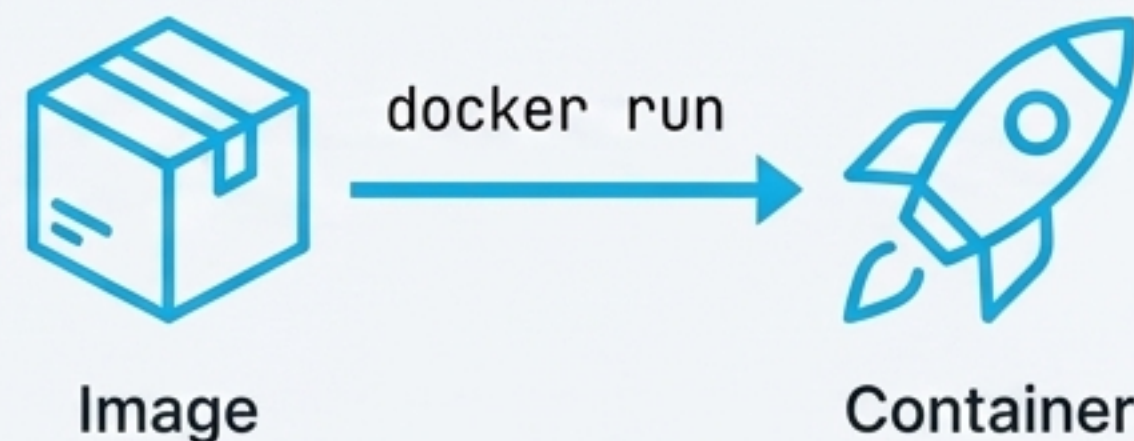
Utilisons notre nouvelle image pour démarrer un conteneur.

La Commande de Lancement

```
docker run -d --name mon-php -p 8080:80 mon-php-image
```

Explication des options

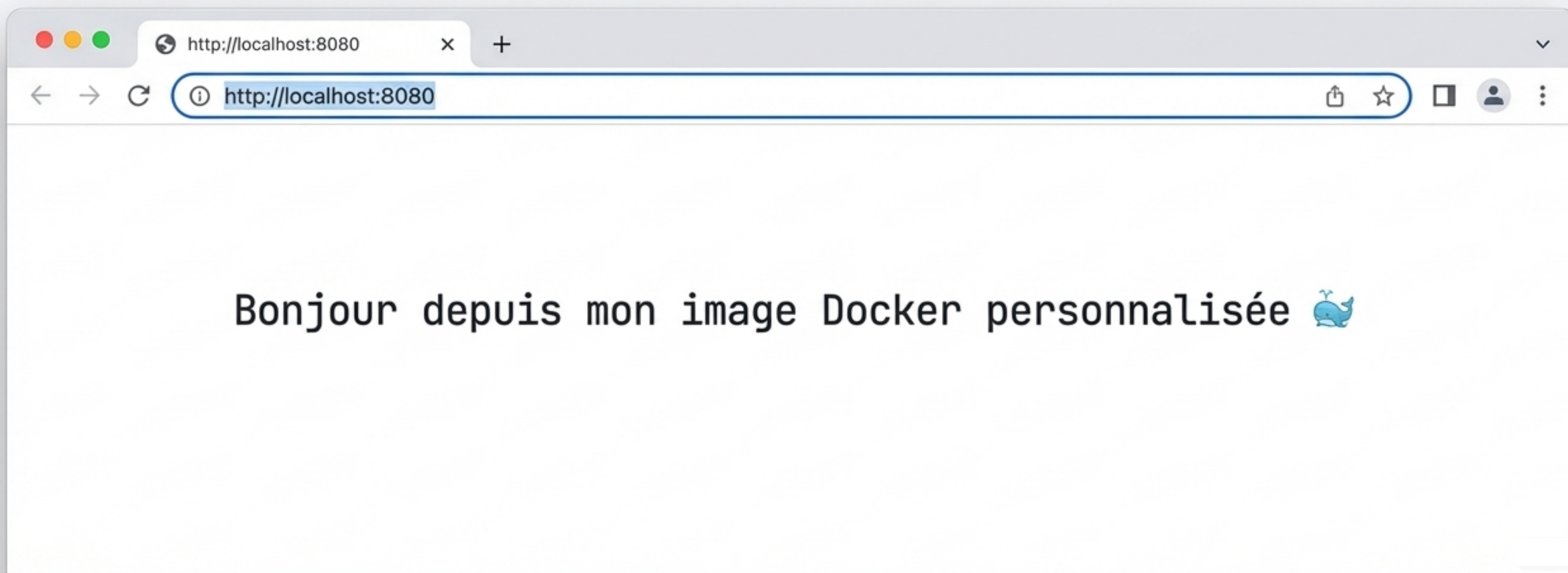
- `-d` : Mode "détaché" (le conteneur tourne en arrière-plan).
- `--name mon-php` : Donne un nom explicite à notre conteneur.
- `-p 8080:80` : Mappe le port 8080 de notre machine hôte vers le port 80 (Apache) du conteneur.
- `mon-php-image` : Le nom de l'image que nous voulons exécuter.



Le Moment de Vérité

Ouvrez votre navigateur et visitez :

<http://localhost:8080>



Inspecter le Conteneur en Marche

Quelques commandes utiles pour vérifier que tout fonctionne comme prévu.



Voir les conteneurs
actifs

```
docker ps
```



Lister les fichiers
dans le conteneur

```
docker exec -it mon-php  
ls /var/www/html
```

(Vous devriez voir `index.php`)



Trouver l'adresse IP
interne du conteneur

```
docker exec -it mon-php  
ip a
```

*(Utile pour le débogage réseau
entre conteneurs)*

La Boucle d'Itération : Mettre à Jour l'Application

Le Scénario : Notre application évolue. Nous devons déployer une nouvelle version.

1. Modifier

Étape 1 : Modifier le code source sur la machine hôte.

```
echo "<?php echo '<h1>Version 2 🚀 </h1>';  
?>" > index.php
```



2. Reconstruire

Étape 2 : Reconstruire l'image avec les changements.

```
docker build -t mon-php-image .
```

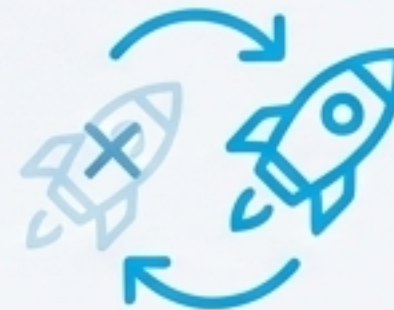


3. Remplacer

Étape 3 : Remplacer l'ancien conteneur par le nouveau.

```
# Supprimer l'ancien conteneur  
docker rm -f mon-php
```

```
# Lancer un nouveau conteneur avec la même  
commande  
docker run -d --name mon-php -p 8080:80 mon-  
php-image
```



✅ **Vérification** : Rafraîchissez `http://localhost:8080`. Vous devriez voir le message "Version 2 🚀".

Garder un Environnement Propre

Une fois le travail terminé, voici comment supprimer les ressources créées.

La Séquence de Nettoyage

1.  Arrêter le conteneur en cours

```
docker stop mon-php
```

2.  Supprimer le conteneur (qui n'est plus en cours d'exécution)

```
docker rm mon-php
```

3.  Supprimer l'image (qui n'est plus utilisée by aucun conteneur)

```
docker image rm mon-php-image
```


Notre Modèle Mental, Solidifié



La Recette
(`Dockerfile`)

Un fichier texte qui décrit les étapes.

Notre exemple : Le fichier ``Dockerfile`` avec les instructions ``FROM``, ``RUN``, ``COPY``, ``CMD``.



Le Modèle
Durable
(`Image`)

Un artefact immuable, contenant l'application.

Notre exemple : L'image ``mon-php-image`` créée avec ``docker build``.



L'Instance
Éphémère
(`Conteneur`)

L'exécution de l'image. Peut être arrêté, supprimé, être arrêté, supprimé, remplacé.

Notre exemple : Le conteneur ``mon-php`` lancé avec ``docker run``.

Mission Accomplie & Prochain Horizon

Checklist de vos nouvelles compétences

-  Le `Dockerfile` est créé et compris.
-  L'image est construite avec succès.
-  Le conteneur affiche la page PHP attendue.
-  Le cycle de mise à jour (modifier -> reconstruire -> relancer) est maîtrisé.
-  La différence fondamentale entre `Dockerfile`, `Image` et `Conteneur` est claire.

Bravo ! Vous maîtrisez maintenant la création d'une image Docker complète. 🎉 🐳

Prochaine Étape : TP 3

Maintenant que nous savons construire une image, comment orchestrer plusieurs conteneurs (par exemple, un service PHP et une base de données MySQL) ? La réponse : `docker-compose`.

